



The Brilliant Employee · 07

My scoreboard printed zero. The wins were real.

A reporting tool assumed every model began with four free wins. The live system had granted them to one model only, so an arm holding 3.0 became 3.0 minus 4 and got clamped to zero. The house rule caught it on 2026-07-26 with three sentences to go before publication: 79.3% typed, 86.4% re-derived, neither quotable.

sgnk.ai/brilliant ↗

Sagnik Mitra

Why this exists

A language model writes a large share of my production code. Wrong work reads exactly like right work, and no prompt fixes that. So the system below exists to find out when the answers were wrong.

From one week, none of it visible from inside the tool: a sizing step ignored on 796 of 816 tasks, and two fixes written up as done that were never made.

THE SYSTEM, AS OF 7 AUGUST 2026, 22:55Z	
rules written	11 June, 57 days ago
ledger running	27 June, 41 days
tasks in the ledger	3,140, across 3,680 rows
controls on tool calls	6 registered, 4 that can refuse

You can rent the model. You cannot rent the part that tells you it was wrong.

Five parts, and where today sits

Five parts sit around the model. Each post takes one of them.

sizing check	sizes a job before a model is picked
model picker	recommends which model gets the job
safety stops	refuse the irreversible, before it runs
grader	grades finished work pass or fail
task log	at least one append-only line per task

Listed in the order they act on a task, not the order they were built.

Today cuts across more than one, so no row is marked.

ASIDE. Only the safety stops can refuse. The other four observe: one advises, one recommends, one grades after the fact, one records.

This page is the map. Nothing here
needs another post to make sense.

Two programs share one scoreboard file

Inside this workplace, the model picker is the part of the system that recommends which AI model gets each job.

Two programs touch that scoreboard, and they were built in different sessions, by the same distracted manager. Me.

the live system

writes the scoreboard, one task at a time

the reporting tool

reads it and prints the headline win rates I actually look at

Plain words: ARM. one model's running score: successes and failures, plus a starting assumption called a prior

This story is about the day I learned they were keeping different scoreboards.

Every scoreboard has a writer and a reader. They can disagree.

The reporting tool guessed the head start

The live system seeds each arm deliberately, and not equally.

THE LIVE SYSTEM WRITES

cheap model

**roughly four
assumed
wins**

expensive model

**one nominal
win, one
nominal loss**

THE REPORTING TOOL ASSUMED

cheap model

**the same
four-win head
start**

expensive model

**the same
four-win head
start**

The reporting tool assumed one rulebook for everyone. So before printing real wins, it subtracted four. From every arm.

A constant in the reader that mirrors a value in the writer will drift.

Three minus four printed as zero real wins

The expensive model's arm sat at a tally of 3.0. The reporting tool subtracted its imagined head start of 4 and got minus 1.

```
pa, pb = prior
a = float(arm.get("a", pa))
b = float(arm.get("b", pb))
return max(0.0, a - pa), max(0.0, b - pb)
```

- 1 arm tally reads 3.0
- 2 reader subtracts an assumed 4-win head start
- 3 3.0 minus 4 = minus 1
- 4 $\max(0, \text{minus } 1) = 0$
- 5 printed: zero real wins

Every success the expensive model had earned was erased before a human ever saw it. Not one bad night. Every report, silently, for as long as both programs existed.

Clamping to zero turns a math error into a clean-looking lie.

The rule fired three sentences before publishing

2026-07-26. I am writing the audit that is supposed to be the honest one, and the headline is already typed: the pooled scoreboard reads 79.3%, nine units of evidence. House rule: re-derive every headline number at write time. So I re-derived it.

ABOUT TO PUBLISH	RE-DERIVED, PRIORS MATCHED
pooled win rate 79.3%	pooled win rate 86.4%
interval 55.9 to 95.2	interval 69.6 to 97.0
weighted evidence 9	weighted evidence 17

Eight units of evidence had been vanishing from every report. Worth being exact about the unit: every point on these six arms came from offline eval runs carried at half weight, so 17 is weighted evidence, not seventeen live jobs. Note the direction: the bug did not flatter the system. It made the expensive model look worse than it was.

Re-derive the headline at write time, from the producer's own files.

A fifth of the fixed number is assumption

The corrected figure reproduces from the audit's own line: 15.0 successes, 2.0 failures, all of it offline eval evidence at half weight, plus the one legitimate prior. $(15 + 4) / (17 + 5) = 86.4\%$. Show the work, or the number is a rumor with decimals.

21%

of the corrected numerator is prior, not evidence

THE AUDIT'S OWN LINE	
successes / failures	15.0 / 2.0
$(15 + 4) / (17 + 5)$	86.4%

That footnote matters. Even the fixed figure is one-fifth assumption by mass, pooled across six arms that are not really comparable to each other. A number can be correctly computed and still be soft in the middle. The audit says so in the same breath it prints it.

Say how much of a figure is measured and how much is prior.

The number improved and stayed unquotable

The house abstention rule: an arm needs at least 5 real observations before its win rate means anything. $\text{MIN_N} = 5$, and there is no exception for numbers that flatter.

0

6

arms that had cleared the five-observation floor on 2026-07-26

So the correction produced a strange artifact: a number that got better AND stayed unreportable. The audit published it anyway, wearing its caveat in the same sentence: 86.4%, and no single arm is quotable yet. That sentence, the improvement and the abstention riding together, is the whole discipline in miniature.

ASIDE. Both figures are pooled across all six arms; neither is one model's record.

A correction can raise the number without earning it a headline.

A hand-carried number cannot notice it is wrong

Two programs exchanging data through a file is the cheapest integration there is, and the most quietly dangerous. The reader must accept what the writer actually writes.

1 same field names

2 same units

3 same priors

A missing key is UNKNOWN, never coerced to zero. And the rule I wrote down: any number bound for a headline gets re-derived by a script at write time, and that script names the file and the key it read. The reporting tool here still does neither.

REJECTED. A dashboard that caches yesterday's computed headline. If the number is not re-derived on read, it is a screenshot, not a measurement.

The reader's assumptions must match the writer's. Check, do not trust.



Re-derive every headline number at write time.

A hardcoded head start erased every win before I saw it. The full arithmetic and the audit trail.

sgnk.ai/brilliant ↗

Sagnik Mitra

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

sgnk.ai · in/sagnikmitra · github.com/sagnikmitra